D-Pathway — Technical Report

A project marketplace that connects Kigali graduates with companies.

1 Scope alignment

On D-Pathway a company posts a short paid job, called a brief. Graduates apply, do the work, and submit it. The company gives each submission a **score from 1 to 5** and picks a winner, who is **paid from money the company set aside** when it posted the brief. Graduates end up with paid work they can show to employers, and companies get to see how someone actually works before they hire.

Everything in the original scope was built. The table lists each planned feature and what it became.

Feature	What was built	Status
Post and manage briefs	A brief moves through draft, published, in progress, and closed. It can be edited or deleted, but not once a winner has been picked.	Done
Find and apply	A catalogue with filters for skill, prize, and company, plus a shortcut to briefs that match the graduate's own skills. Briefs you have applied to drop off the list so you cannot apply twice.	Done
Submit and evaluate	Graduates submit a link, a file, or a repository. The company gives a score, written feedback, and a decision. A re-submission flags the old score so it gets reviewed again.	Done
Prizes and payment	Companies top up a wallet with Stripe. The prize is set aside when the brief is created and paid to the winner automatically. The winner can withdraw it.	Done
Talent search	Companies search graduates by skill and see real numbers on each profile: briefs finished, average score, and prizes won.	Done
Ratings	Companies rate submissions, and graduates rate companies. A company's rating shows on its public page.	Done
Candidate board	A board shows where each applicant stands: applied, accepted, submitted, evaluated, or winner.	Done
Notifications	An in-app activity feed, plus email (via Resend) when an application is accepted or declined, work is submitted, a score is given, or a prize is awarded.	Done
Profiles	Graduates have an editable profile with a photo and a public page they can share. Companies have a profile with a logo and a public page.	Done

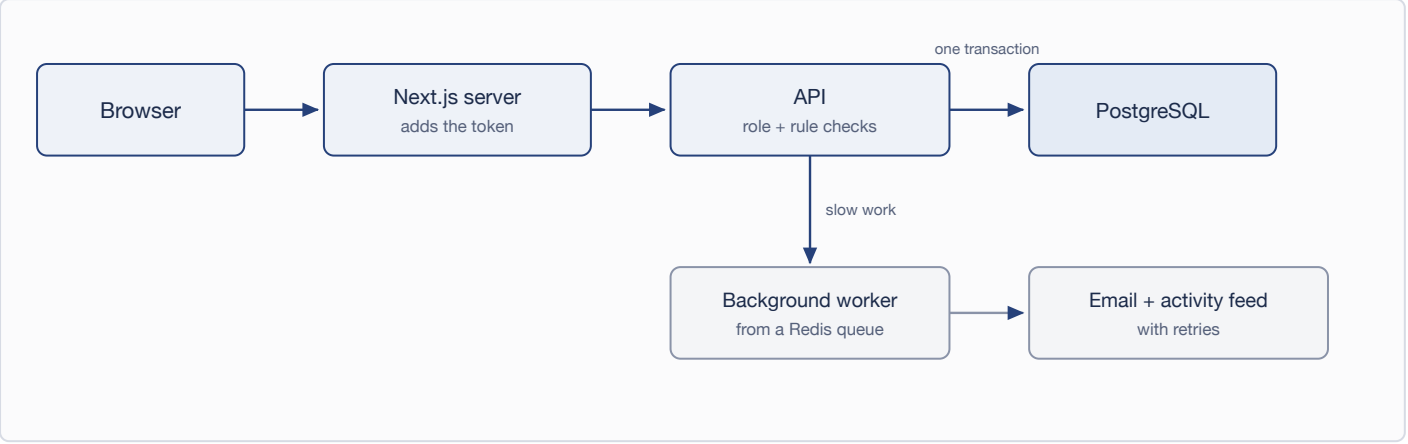
2 Technical functionality

Backend	
Framework	NestJS, split into modules
Database	PostgreSQL through TypeORM
Migrations	Written by hand
Background	Redis + BullMQ jobs
Cache	Redis, for the catalogue
Payments	Stripe
Email	Resend
Uploads	Cloudinary

Frontend	
Framework	Next.js (App Router)
UI	React with Tailwind
Login	Token in an httpOnly cookie
Data	React Query
Rendering	Server-rendered pages
Fonts	Self-hosted
Routing	Grouped by role
Layout	Mobile drawer, desktop sidebar

How a request flows

The browser never holds the login token. The Next.js server keeps it in a cookie and adds it to each call to the API. The API checks the user's role and the business rules, then reads or writes the database. Slow work, like sending email, is handed to a background worker so the request can finish quickly.



One request: the browser talks only to the Next.js server, which forwards to the API. Email is sent off the main path.

A few design choices

- **Prize money.** When a company sets a prize, the amount is taken from its wallet and marked as held. When it picks a winner, that money moves to the winner's wallet. Both steps happen in **one database transaction**, so a balance is never left half-changed.
- **Email off the main path.** The API does not send email while answering a request. It puts an event on a queue, and a worker sends the email and writes the activity feed. Requests stay fast, and a failed email can retry.
- **The candidate board.** Each applicant's stage is worked out from data that already exists, instead of storing a separate status that could fall out of step.

3 Testing approaches and results

There are two sets of tests, one for the backend and one for the frontend. Each build checks coverage and fails if it drops below **80 percent**. In total **524 tests pass**.

Backend (299 tests)

Every service and rule is tested on its own. The database and the outside services (Stripe, Resend, Cloudinary, Redis) are replaced by stand-ins, so the tests are fast and do not touch the network. This covers the parts most likely to break: deadlines, status changes, team-size limits, and the prize math. Line coverage of the services and rules is **about 99 percent**. Writing the tests also settled a few edge cases, such as not letting an application be accepted after its deadline, and marking an old score for review when work is re-submitted.

Frontend (225 tests)

Components, hooks, and helpers are tested with React Testing Library. These check what the backend tests cannot: loading and error states, form validation, and button actions like bookmarking a brief, rating a company, and logging out. Line coverage of the library and component code is **about 99 percent**.

Performance

Page speed was measured on a production build with Playwright, on a fast machine and on a throttled slow one (four times slower CPU and a 3G network). The target for Largest Contentful Paint is **2.5 seconds**.

Page	First paint (ms)	LCP, fast (ms)	LCP, slow (ms)	Result
Dashboard	68	156	760	Good
Catalogue	112	308	820	Good
Landing	228	164	5,476	Slow on 3G

The app pages stay fast even on the slow profile. The landing page is slow on 3G because of large photos. That is an image problem, not a code one, and the fix is smaller image formats and lazy loading. The pages render correctly in Chrome, Firefox, and Safari, and at both phone and desktop widths.

4 Code quality

Structure. The backend is split into 16 small modules (auth, projects, wallet, talent, and so on). Each has its own controller, service, entity, and data objects. Controllers only handle the HTTP part, services hold the logic, and entities describe the data. A shared base class gives every table its id, timestamps, and soft delete. On the frontend, pages are grouped by role, and small shared pieces (button, card, badge, tag input) are reused across the app.

Types. TypeScript is used on both sides, and the API response types are shared with the frontend so the two stay in agreement.

Validation and documentation. Incoming data is checked against typed rules, and those same rules describe the API. Database changes are written by hand, which keeps them easy to read and review.

5 Summary

Everything in scope was built and tested. There are **524 tests in all**, **299** on the backend and **225** on the frontend, and they all pass, both sides **above 99 percent coverage**. The app pages meet the speed target, and the site works across the main browsers and screen sizes. The money handling, which is the riskiest part, runs inside **database transactions** and has its own tests. The one thing left is smaller images on the landing page.